



UIDE

Powered by
Arizona State University®

1

UNIVERSIDAD INTERNACIONAL DEL ECUADOR - UIDE



INGENIERIA EN CIBERSEGURIDAD

ONLINE

PRIMER SEMESTRE

LOGICA DE PROGRAMACION

PROYECTO INTEGRADOR

AUTORES:

GOMEZ SERRANO CHRISTOPHER STALIN

DOCENTE:

ING. LILIAN MARLENE AMAN RAMOS

MAYO – NOVIEMBRE 2026

QUITO – ECUADOR

Generador de Contraseñas Seguras

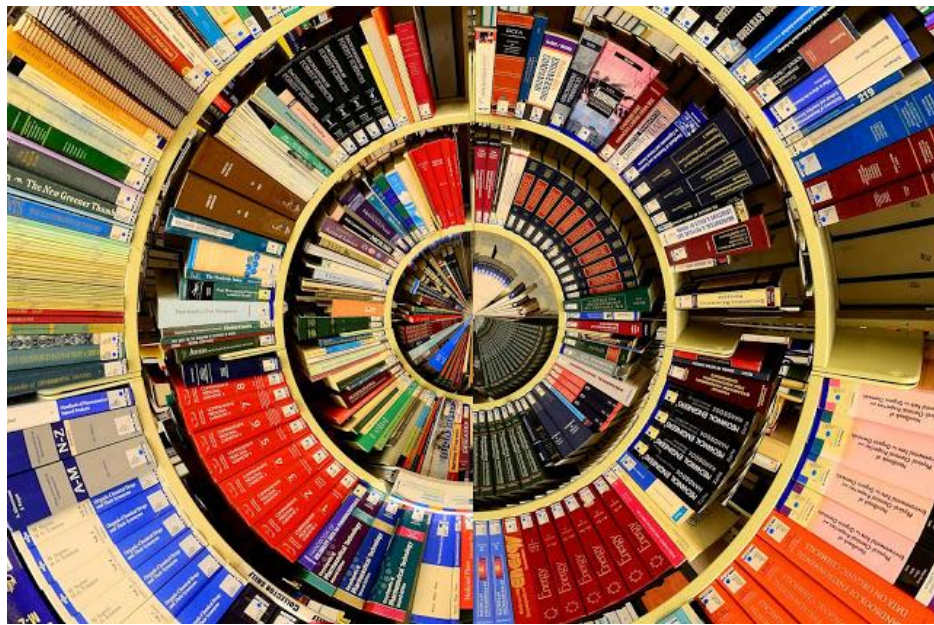
Se eligió realizar este proyecto ya que se vio los problemas que existían en la actualidad como son el robo de información mayormente debido a la baja seguridad que como usuarios les damos a nuestras contraseñas por este motivo se pensó una manera de poder ayudar al usuario común que muchas veces no tienen los conocimientos suficientes para una integración de seguridad optima, haciendo un sistema simple y fácil de entender para todo tipo de usuarios y edades.

Librerías

En este sistema se usaron dos librerías la primera llamada **Random** que esta lo que hace es que en un arreglo de caracteres nos selecciona un valor al azar de nuestro arreglo logrando así que los valores sean de manera aleatoria y que no sigan un flujo simple de creación de cadenas.



La segunda librería tiene como nombre **String** esta guarda distintos valores en distintas clases como los son de tipo numérico y de tipo alfanumérico guardando todas las letras del abecedario en mayúscula y minúscula con esto logramos optimizar líneas de código teniendo ya todos los datos almacenados en diferentes clases de la librería. La desventaja de esta librería es que no obtiene clases de tipo carácter especiales.

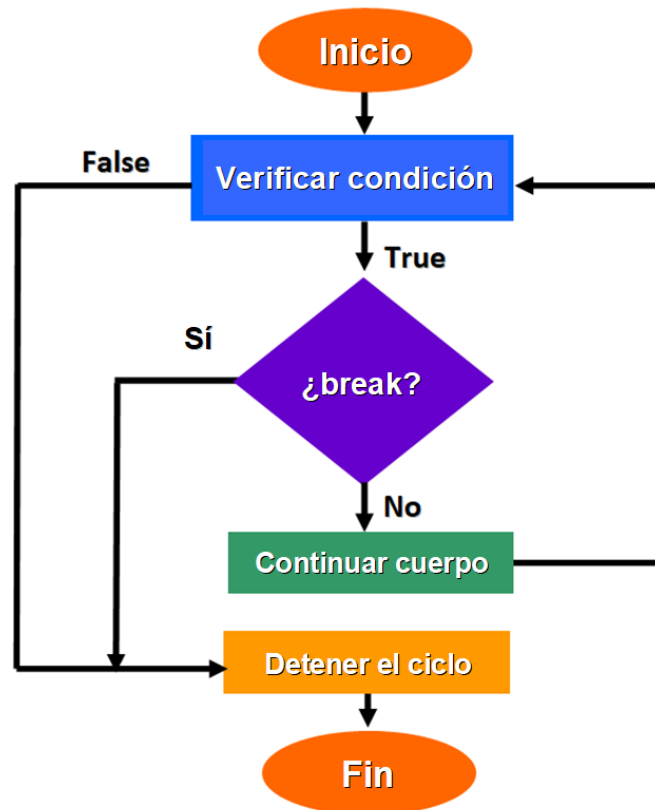


Arreglos

Se uso arreglos en el sistema para cubrir ciertas desventajas que nos dan las librerías como son los datos de tipo carácter especial. Creando así un arreglo de los caracteres especiales mas comunes usados en sistemas varios.

Ciclos

Se implemento ciclos en el sistema para que no sea un sistema tan directo y obsoleto logrando que sea más amigable con el usuario final, en este caso se implemento bucles while y for para los diferentes ciclos, también así ahorrando líneas de código.



Condicionales

Se implementaron condicionales para diferentes validaciones logrando blindar posibles malos usos de los usuarios y mala obtención de información.

```

if Condición :
    #código
    #código
elif Condición :
    #código
    #código
else:
    #código
    #código
#código fuera del if/else
  
```

Obtención de excepciones.

Se implemento excepciones para lograr capturar errores en tiempo de ejecución ya sean por un mal ingreso de datos o un bug generado por el usuario permitiendo que el sistema no solo se cierre si no que se notifique lo que se realizó mal.



Resumen sistema

Importación de librerías

Lo primero que hace el programa es cargar los módulos necesarios en la memoria. Esto se hace con dos líneas muy sencillas:

```
import random  
  
import string
```

El módulo aleatorio le dice a Python: "cuando te pida que elijas algo aleatorio, usa este módulo". El módulo de cadena, por otro lado, evita que el programador tenga que escribir a mano toda la lista de alfabetos y números.

Definición de arreglos.

A continuación, se construyen cuatro variables que definen el universo de caracteres disponibles:

```
LETRAS      = string.ascii_letters  
  
NUMEROS     = string.digits  
  
ESPECIALES  = "!@#$%^&*()-_+=[]{}|;:,.<>?"  
  
TODOS       = LETRAS + NUMEROS + ESPECIALES
```

Las primeras tres variables agrupan cada categoría de personaje por separado. El cuarto - TODOS - es simplemente una concatenación de los anteriores y es el conjunto completo del que se derivarán los caracteres que complementarán la contraseña. Escribir constantes grandes es una buena práctica en Python porque indica visualmente que estos valores no cambiarán en tiempo de ejecución.

Interacción con el usuario

En esta etapa el programa se comunica con la persona que lo está usando. Primero muestra un encabezado y pregunta el nombre:

```
nombre = input("\nHola! ¿Cuál es tu nombre? ").strip()

if nombre == "": nombre = "Usuario"
```

El método `.strip()` elimina los espacios que el usuario haya escrito antes o después de su nombre, lo cual es pequeño, pero mejora la presentación. Si el campo se deja en blanco, el programa falla, pero simplemente asigna el nombre genérico "Usuario". Esto hace que el programa sea tolerante al comportamiento de diferentes usuarios.

Luego viene la solicitud de la longitud, que es el fragmento más elaborado del código:

```
while True:

    try:

        largo = int(input("¿Cuántos caracteres? (mínimo 8): "))

        if largo < 8:

            print("Debe tener al menos 8 caracteres.")

        else:

            break

    except ValueError:

        print("Por favor ingresa un número válido.")
```

Este bucle es un ejemplo clásico de validación sólida. El programa no continúa hasta que el usuario ingresa un número entero igual o mayor que 8. Si escribe texto (como "diez"), el bloque de excepción detecta el error antes de que el programa se interrumpa y muestra un mensaje amigable. Si ingresa un número menor a 8, se muestra un mensaje de condición interna y se deben solicitar los datos nuevamente.

Generación de contraseña

Ésta es la esencia del programa. La generación se lleva a cabo en tres etapas:

Primero, se garantiza que la contraseña contenga al menos un carácter de cada una de las categorías requeridas:

```
obligatorios = [
    random.choice(LETRAS),
    random.choice(NUMEROS),
    random.choice(ESPECIALES),
]
```

Esto resuelve un gran problema: si elegimos todos los caracteres de forma completamente aleatoria, existiría una posibilidad (por pequeña que sea) de que la contraseña termine sin números ni símbolos. Al presionar estos tres caracteres, esta opción desaparecerá. Luego se generan el resto de caracteres, tomados de todo el conjunto TODOS:

```
resto = [random.choice(TODOS) for _ in range(largo - len(obligatorios))]
```

Esta línea usa una comprensión de listas, una de las características más elegantes de Python: genera tantos elementos como se necesiten para llegar al largo solicitado, descontando los tres obligatorios ya incluidos.

Finalmente, se mezclan todos los caracteres de forma aleatoria y se ensambla la contraseña:

```
contrasena_lista = obligatorios + resto
random.shuffle(contrasena_lista)
contrasena = "".join(contrasena_lista)
```


El shuffle es crucial: sin él, la contraseña siempre comenzaría con una letra, luego un número y luego un símbolo (se requieren tres), introduciendo un patrón predecible. Cuando se barajan, los imperativos se dividen en posiciones completamente aleatorias.

Presentación de contraseña

Una vez generada la contraseña, el programa la muestra de forma clara y bien organizada, acompañada de información útil para el usuario:

```
print("=" * 45)

print(f"  Tu contraseña nueva es:")

print(f"  {contrasena}")

print("=" * 45)

print(f"Con un largo      : {largo} caracteres")

print(";Guárdala en un lugar seguro!")
```

El uso de f-strings (cadenas de formato) permite insertar el valor de las variables directamente dentro del texto, lo que hace el código más limpio y legible que la concatenación tradicional con el operador

Pruebas de ejecución

Se realizaron varias pruebas para verificar la funcionalidad del programa en diferentes situaciones. En general, el sistema respondió correctamente tanto ante datos válidos como inválidos: generando contraseñas seguras cuando la información era correcta, asignando un nombre predeterminado cuando el usuario no ingresó ninguno y validando correctamente la longitud mínima para evitar continuar con valores ilegales. Además, manejaba errores cuando se ingresaban datos incorrectos, como texto en lugar de números, sin provocar que el programa fallara.

```
Hola! ¿Cuál es tu nombre? Cristo
```

```
Bienvenido, Cristo!
```

```
¿Cuántos caracteres quieres para tu contraseña? (mínimo 8): 7  
La contraseña debe tener al menos 8 caracteres. Intenta de nuevo.
```

```
¿Cuántos caracteres quieres para tu contraseña? (mínimo 8): 9
```

```
=====
```

```
Tu contraseña nueva es:
```

```
.rcR5)[Ht
```

```
=====
```

```
Con un largo      : 9 caracteres
```

```
Contiene : letras, números y símbolos
```

```
=====
```

```
¡Guárdala en un lugar seguro!
```

Conclusiones

La validación de datos es tan importante como la lógica subyacente. Un programa que no maneja entradas incorrectas no es un programa completo. Es un programa que fracasa en el momento menos oportuno.

Python ofrece herramientas muy poderosas con poco código. La combinación de `random.choice()`, `random.shuffle()` y comprensión de listas le permite crear un generador seguro en solo unas pocas líneas, lo que requeriría mucho más trabajo en otros lenguajes.

La seguridad digital comienza con hábitos simples. Usar contraseñas largas, aleatorias y únicas para cada cuenta es la forma más efectiva en que una persona puede proteger su información. Este programa facilita precisamente eso.

El diseño centrado en el usuario es importante incluso en proyectos pequeños. Detalles como personalizar el saludo con un nombre o redirigir al usuario cuando se equivoca hacen que la experiencia sea más agradable y el programa más profesional.

Los conceptos aprendidos en clase tienen aplicación real e inmediata. Las estructuras de control (`if`, `while`, `try/except`) no son ejercicios abstractos. Son las herramientas que hacen que los programas realmente funcionen en el mundo real.